P3508R0
Wording for "constexpr for specialized memory algorithms"

Published Proposal, 2024-11-19

Author:
Giuseppe D'Angelo

Audience:
LWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper provides an updated wording to [P2283R2] ("constexpr for specialized memory algorithms").

Table of Contents

References

Informative References

§ 1. Changelog

- R0
  - First submission

§ 2. Motivation and scope

Please see [P2283R2] ("constexpr for specialized memory algorithms") for the general motivation for these changes.

The wording in [P2283R2] did two things:

1. it added `constexpr` to most uninitialized memory algorithms;
2. it changed the implementation of them to use `std::construct_at` instead of a call to `operator new`.

With the adoption of [P2747R2] ("`constexpr` placement new") in the core language the latter change is no longer needed, and it's actually counter-productive as it impedes value elision (for instance, in case the input iterators to these algorithms return prvalues).

Therefore, this paper proposes an updated wording where we just add `constexpr` to the algorithms' signatures, as that's both necessary and sufficient to achieve the stated goal of being able to use the algorithms from constant evaluation (and does not pessimize elision).

For the `uninitialized_default_construct` family of algorithms please see [P3369R0].

§ 3. Impact on the Standard

This proposal is a pure library addition. The necessary changes to core language have already been adopted.

§ 4. Proposed Wording

All the proposed changes are relative to [N4993].

§ 4.1. Feature-testing macro

In [version.syn], modify the value of the `__cpp_lib_raw_memory_algorithms` to match the date of adoption of the present proposal.

§ 4.2. Wording

Modify [memory.syn] as shown:

```
template<class NoThrowForwardIterator>
  constexpr
  void uninitialized_value_construct(NoThrowForwardIterator first,
                                     NoThrowForwardIterator last);
template<class ExecutionPolicy, class NoThrowForwardIterator>
  void uninitialized_value_construct(ExecutionPolicy&& exec,
                                     NoThrowForwardIterator first,
                                     NoThrowForwardIterator last);
template<class NoThrowForwardIterator, class Size>
  constexpr
  NoThrowForwardIterator
    uninitialized_value_construct_n(NoThrowForwardIterator first, Size n);
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size>
  NoThrowForwardIterator
    uninitialized_value_construct_n(ExecutionPolicy&& exec,
                                    NoThrowForwardIterator first, Size n);

namespace ranges {
  template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
    requires default_initializable<iter_value_t<I>>
    constexpr
      I uninitialized_value_construct(I first, S last);
  template<nothrow-forward-range R>
    requires default_initializable<range_value_t<R>>
      constexpr
      borrowed_iterator_t<R> uninitialized_value_construct(R&& r);
```

```
  template<nothrow-forward-iterator I>
    requires default_initializable<iter_value_t<I>>
      constexpr
      I uninitialized_value_construct_n(I first, iter_difference_t<I> n);
}

template<class InputIterator, class NoThrowForwardIterator>
  constexpr
  NoThrowForwardIterator uninitialized_copy(InputIterator first,
                                            InputIterator last,
                                            NoThrowForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator, class NoThrowForward
  NoThrowForwardIterator uninitialized_copy(ExecutionPolicy&& exec,
                                            ForwardIterator first, Forward
                                            NoThrowForwardIterator result);
template<class InputIterator, class Size, class NoThrowForwardIterator>
  constexpr
  NoThrowForwardIterator uninitialized_copy_n(InputIterator first, Size n,
                                              NoThrowForwardIterator result
template<class ExecutionPolicy, class ForwardIterator, class Size,
         class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_copy_n(ExecutionPolicy&& exec,
                                              ForwardIterator first, Size n
                                              NoThrowForwardIterator result

namespace ranges {
  template<class I, class O>
    using uninitialized_copy_result = in_out_result<I, O>;
  template<input_iterator I, sentinel_for<I> S1,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
    requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
      constexpr
      uninitialized_copy_result<I, O>
        uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast);
  template<input_range IR, nothrow-forward-range OR>
    requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
      constexpr
      uninitialized_copy_result<borrowed_iterator_t<IR>, borrowed_iterator_
        uninitialized_copy(IR&& in_range, OR&& out_range);

  template<class I, class O>
    using uninitialized_copy_n_result = in_out_result<I, O>;
  template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-
```

```
    requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
      constexpr
      uninitialized_copy_n_result<I, O>
        uninitialized_copy_n(I ifirst, iter_difference_t<I> n,
                             O ofirst, S olast);
}

template<class InputIterator, class NoThrowForwardIterator>
  constexpr
  NoThrowForwardIterator uninitialized_move(InputIterator first,
                                            InputIterator last,
                                            NoThrowForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator, class NoThrowForward
  NoThrowForwardIterator uninitialized_move(ExecutionPolicy&& exec,
                                            ForwardIterator first, Forward
                                            NoThrowForwardIterator result);
template<class InputIterator, class Size, class NoThrowForwardIterator>
  constexpr
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n,
                         NoThrowForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator, class Size,
         class NoThrowForwardIterator>
  pair<ForwardIterator, NoThrowForwardIterator>
    uninitialized_move_n(ExecutionPolicy&& exec,
                         ForwardIterator first, Size n, NoThrowForwardItera

namespace ranges {
  template<class I, class O>
    using uninitialized_move_result = in_out_result<I, O>;
  template<input_iterator I, sentinel_for<I> S1,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
    requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>
      constexpr
      uninitialized_move_result<I, O>
        uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);
  template<input_range IR, nothrow-forward-range OR>
    requires constructible_from<range_value_t<OR>, range_rvalue_reference_t
      constexpr
      uninitialized_move_result<borrowed_iterator_t<IR>, borrowed_iterator_
        uninitialized_move(IR&& in_range, OR&& out_range);

  template<class I, class O>
```

```
    using uninitialized_move_n_result = in_out_result<I, O>;
  template<input_iterator I,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
    requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>
      constexpr
      uninitialized_move_n_result<I, O>
        uninitialized_move_n(I ifirst, iter_difference_t<I> n,
                             O ofirst, S olast);
}

template<class NoThrowForwardIterator, class T>
  constexpr
  void uninitialized_fill(NoThrowForwardIterator first,
                          NoThrowForwardIterator last, const T& x);
template<class ExecutionPolicy, class NoThrowForwardIterator, class T>
  void uninitialized_fill(ExecutionPolicy&& exec,
                          NoThrowForwardIterator first, NoThrowForwardItera
                          const T& x);
template<class NoThrowForwardIterator, class Size, class T>
  constexpr
  NoThrowForwardIterator
    uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T& x);
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size, c
  NoThrowForwardIterator
    uninitialized_fill_n(ExecutionPolicy&& exec,
                         NoThrowForwardIterator first, Size n, const T& x);

namespace ranges {
  template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S, class T>
    requires constructible_from<iter_value_t<I>, const T&>
      constexpr
      I uninitialized_fill(I first, S last, const T& x);
  template<nothrow-forward-range R, class T>
    requires constructible_from<range_value_t<R>, const T&>
      constexpr
      borrowed_iterator_t<R> uninitialized_fill(R&& r, const T& x);

  template<nothrow-forward-iterator I, class T>
    requires constructible_from<iter_value_t<I>, const T&>
      constexpr
```

```
    I uninitialized_fill_n(I first, iter_difference_t<I> n, const T& x);
}
```

Modify [uninitialized.construct.value] as shown:

```
template<class NoThrowForwardIterator>
  constexpr
  void uninitialized_value_construct(NoThrowForwardIterator first,
                                     NoThrowForwardIterator last);
```

```
namespace ranges {
  template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
    requires default_initializable<iter_value_t<I>>
    constexpr
      I uninitialized_value_construct(I first, S last);
  template<nothrow-forward-range R>
    requires default_initializable<range_value_t<R>>
      constexpr
      borrowed_iterator_t<R> uninitialized_value_construct(R&& r);
}
```

```
template<class NoThrowForwardIterator, class Size>
  constexpr
  NoThrowForwardIterator
    uninitialized_value_construct_n(NoThrowForwardIterator first, Size n);
```

```
namespace ranges {
  template<nothrow-forward-iterator I>
    requires default_initializable<iter_value_t<I>>
      constexpr
      I uninitialized_value_construct_n(I first, iter_difference_t<I> n);
}
```

Modify [uninitialized.copy] as shown:

```
template<class InputIterator, class NoThrowForwardIterator>
  constexpr
  NoThrowForwardIterator uninitialized_copy(InputIterator first,
                                            InputIterator last,
                                            NoThrowForwardIterator result);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S1,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
    requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
      constexpr
      uninitialized_copy_result<I, O>
        uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast);
  template<input_range IR, nothrow-forward-range OR>
    requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
      constexpr
      uninitialized_copy_result<borrowed_iterator_t<IR>, borrowed_iterator_
        uninitialized_copy(IR&& in_range, OR&& out_range);
}
```

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
  constexpr
  NoThrowForwardIterator uninitialized_copy_n(InputIterator first, Size n,
                                              NoThrowForwardIterator result
```

```
namespace ranges {
  template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-
    requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
      constexpr
      uninitialized_copy_n_result<I, O>
        uninitialized_copy_n(I ifirst, iter_difference_t<I> n,
                             O ofirst, S olast);
}
```

✧ ✧ ✧

Modify [uninitialized.move] as shown:

```
template<class InputIterator, class NoThrowForwardIterator>
  constexpr
  NoThrowForwardIterator uninitialized_move(InputIterator first,
                                            InputIterator last,
                                            NoThrowForwardIterator result);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S1,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
    requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>
      constexpr
      uninitialized_move_result<I, O>
        uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);
  template<input_range IR, nothrow-forward-range OR>
    requires constructible_from<range_value_t<OR>, range_rvalue_reference_t
      constexpr
      uninitialized_move_result<borrowed_iterator_t<IR>, borrowed_iterator_
        uninitialized_move(IR&& in_range, OR&& out_range);
}
```

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
  constexpr
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n,
                         NoThrowForwardIterator result);
```

```
namespace ranges {
  template<input_iterator I,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
    requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>
      constexpr
      uninitialized_move_n_result<I, O>
        uninitialized_move_n(I ifirst, iter_difference_t<I> n,
                             O ofirst, S olast);
}
```

✧ ✧ ✧

Modify [uninitialized.fill] as shown:

```
template<class NoThrowForwardIterator, class T>
  constexpr
  void uninitialized_fill(NoThrowForwardIterator first,
                          NoThrowForwardIterator last, const T& x);
```

```
namespace ranges {
  template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S, class T>
    requires constructible_from<iter_value_t<I>, const T&>
      constexpr
      I uninitialized_fill(I first, S last, const T& x);
  template<nothrow-forward-range R, class T>
    requires constructible_from<range_value_t<R>, const T&>
      constexpr
      borrowed_iterator_t<R> uninitialized_fill(R&& r, const T& x);
}
```

```
template<class NoThrowForwardIterator, class Size, class T>
  constexpr
  NoThrowForwardIterator
    uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T& x);
```

```
namespace ranges {
  template<nothrow-forward-iterator I, class T>
    requires constructible_from<iter_value_t<I>, const T&>
      constexpr
      I uninitialized_fill_n(I first, iter_difference_t<I> n, const T& x);
}
```

§ 5. Acknowledgements

All credits for [P2283R2] go to Michael Schellenberger Costa, whose work has also inspired our [P3369R0].

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[N4993]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 16 October 2024. URL: https://wg21.link/n4993

[P2283R2]

Michael Schellenberger Costa. *constexpr for specialized memory algorithms*. 26 November 2021. URL: https://wg21.link/p2283r2

[P2747R2]

Barry Revzin. *constexpr placement new*. 19 March 2024. URL: https://wg21.link/p2747r2

[P3369R0]

Giuseppe D'Angelo. *constexpr for uninitialized_default_construct*. 28 July 2024. URL: https://wg21.link/p3369r0